Automatic Background Music for Videos Using AI-Generated Music

Olivier

2025

Abstract

This article presents a method to automatically add background music to videos using AI-generated music. The process involves extracting a random music track from a directory, generating unique royalty-free music using AI models like MusicGen, and integrating it into a video using FFmpeg. Additionally, we discuss how to fine-tune lightweight models for cost-effective music generation and automate prompt creation using subtitles. This approach provides a complete solution for enhancing videos with custom background music, tailored for social media and other platforms.

1 Introduction

Adding background music to videos is a powerful way to enhance viewer engagement and create a professional atmosphere. However, finding royalty-free music that fits the video's theme can be challenging. This article proposes a solution that combines FFmpeg for video processing, AI-generated music for unique soundtracks, and the previously created auto-subtitling tool for context-aware music generation. The goal is to create a fully automated pipeline that adds custom background music to videos.

2 Technical Setup

To implement this solution, the following tools and libraries are required:

- **FFmpeg**: For video and audio processing.
- **MusicGen**: An AI model for generating music.
- **Maestro Dataset**: For fine-tuning lightweight music models.
- **Auto-Subtitling Tool**: To extract context from video subtitles.

2.1 Installation

Install FFmpeg and other dependencies using the following commands:

```
pip install ffmpeg
conda install -c conda-forge musicgen
```

3 Extracting Random Music from a Directory

To add background music to a video, we first need a music track. The following Python code extracts a random music file from a specified directory and integrates it into a video using FFmpeg.

```
import os
import random
import subprocess

def add_random_music(video_path, music_dir, output_path):
    # Select a random music file from the directory
    music_files = [f for f in os.listdir(music_dir) if f.endswith('.mp3')]
    if not music_files:
        raise ValueError("No music files found in the directory.")
    music_path = os.path.join(music_dir, random.choice(music_files))
    # Use FFmpeg to add music to the video
```

```
    command = [
        'ffmpeg', '-i', video_path, '-i', music_path,
        '-filter_complex', '[0:a][1:a]amerge=inputs=2[a]',
        '-map', '0:v', '-map', '[a]', '-c:v', 'copy', '-ac', '2', output_path
    ]
    subprocess.run(command, check=True)

# Example usage
add_random_music('input_video.mp4', 'music_directory', 'output_video.mp4')
```

4 Generating Music Using AI

To create unique royalty-free music, we use AI models like MusicGen. However, generating high-quality music often requires a GPU and significant computational resources. For cost-effective solutions, we can fine-tune lightweight models using the Maestro dataset.

4.1 Using MusicGen

MusicGen is a state-of-the-art model for music generation. The following steps explain how to use the 'musicgen-small' variant to generate music:

1. Install the required libraries using 'pip':

   ```
   pip install torch transformers soundfile
   ```

2. Load the pre-trained 'musicgen-small' model and its processor:

   ```
   from transformers import MusicgenForConditionalGeneration, MusicgenProcessor

   # Load the pre-trained MusicGen small model
   model = MusicgenForConditionalGeneration.from_pretrained("facebook/musicgen-small")

   # Load the processor for handling inputs and outputs
   processor = MusicgenProcessor.from_pretrained("facebook/musicgen-small")
   ```

3. Define an input prompt for music generation. For example, a text prompt:

   ```
   prompt = "A calm piano melody for a relaxing video."
   ```

4. Generate music using the 'generate' method:

   ```
   import soundfile as sf

   # Generate 10 seconds of music
   audio_values = model.generate(prompt, max_duration=10)

   # Save the generated audio to a file
   sf.write("generated_music.wav", audio_values.cpu().numpy(),
            samplerate=model.config.audio_encoder.sampling_rate)
   ```

5. Customize the generation process by adjusting parameters such as 'top_k', 'top_p', '$temperature$', and'cfg_coef' :

   ```
   audio_values = model.generate(
       prompt,
       max_duration=15,  # Generate 15 seconds of music
       top_k=250,        # Top-k sampling
       top_p=0.95,       # Top-p (nucleus) sampling
       temperature=1.0,  # Randomness level
       cfg_coef=10.0     # Conditioning strength
   )
   ```

```
sf.write("custom_generated_music.wav", audio_values.cpu().numpy(),
         samplerate=model.config.audio_encoder.sampling_rate)
```

This approach allows you to generate high-quality music using the lightweight 'musicgen-small' model, which is well-suited for resource-constrained environments or quick experiments.

4.2 Fine-Tuning Lightweight Models

For cost-effective music generation, we can fine-tune lightweight models using the Maestro dataset. The following steps outline the process:

1. Download the Maestro dataset from MAESTRO Dataset.
2. Convert MIDI files in the dataset to audio format (e.g., WAV). You can use tools like `fluidsynth`:

   ```
   sudo apt-get install fluidsynth
   fluidsynth -ni /path/to/soundfont.sf2 /path/to/midi/file.mid -F /path/to/output/file.wav
   ```

   Alternatively, use Python libraries like `pretty_midi`:

   ```
   import pretty_midi
   import soundfile as sf

   def midi_to_audio(midi_file, output_file):
       midi_data = pretty_midi.PrettyMIDI(midi_file)
       audio_data = midi_data.fluidsynth(fs=44100)
       sf.write(output_file, audio_data, 44100)
   ```

3. Preprocess the audio data by resampling, segmenting, and normalizing it:

   ```
   import librosa
   import os

   def preprocess_audio(input_dir, output_dir, target_sr=16000, segment_length=5):
       if not os.path.exists(output_dir):
           os.makedirs(output_dir)

       for filename in os.listdir(input_dir):
           if filename.endswith('.wav'):
               input_path = os.path.join(input_dir, filename)
               y, sr = librosa.load(input_path, sr=None)
               y = librosa.resample(y, orig_sr=sr, target_sr=target_sr)

               num_segments = len(y) // (segment_length * target_sr)
               for i in range(num_segments):
                   start = i * segment_length * target_sr
                   end = (i + 1) * segment_length * target_sr
                   segment = y[start:end]
                   segment /= np.max(np.abs(segment))

                   output_path = os.path.join(output_dir, f"{filename}_seg{i}.wav")
                   sf.write(output_path, segment, target_sr)
   ```

4. Fine-tune the MusicGen model using the preprocessed dataset:

   ```
   from transformers import MusicgenForConditionalGeneration, MusicgenProcessor, Trainer, Trainin

   # Load pre-trained MusicGen model
   model = MusicgenForConditionalGeneration.from_pretrained("facebook/musicgen-small")
   processor = MusicgenProcessor.from_pretrained("facebook/musicgen-small")

   # Define training arguments
   ```

```
training_args = TrainingArguments(
    output_dir="./results",
    evaluation_strategy="steps",
    eval_steps=500,
    save_steps=1000,
    logging_dir="./logs",
    learning_rate=1e-4,
    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,
    num_train_epochs=3,
    weight_decay=0.01,
)

# Define trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset,
    tokenizer=processor,
)

# Train the model
trainer.train()

# Save the fine-tuned model
model.save_pretrained("./fine_tuned_musicgen")
processor.save_pretrained("./fine_tuned_musicgen")
```

5 Automating Prompt Creation

To generate context-aware music, we can use the video's subtitles as a prompt. Instead of summarizing the entire text, we extract the first sentence from the subtitles.

```
def get_prompt_from_subtitles(subtitle_path):
    with open(subtitle_path, 'r', encoding='utf-8') as file:
        subtitles = file.read()
    first_sentence = subtitles.split('.')[0]  # Extract the first sentence
    return first_sentence

# Example usage
prompt = get_prompt_from_subtitles('subtitles.txt')
music = generate_music(prompt, model_path='piano_model')
music.save('custom_background.mp3')
```

6 Conclusion

This article presented a comprehensive method for adding automatic background music to videos using AI-generated music and FFmpeg. By combining random music extraction, AI music generation, and context-aware prompt creation, we created a cost-effective and customizable solution. This approach is ideal for content creators looking to enhance their videos with unique, royalty-free music.